Nanjing University of Information Science and Technology Laboratory (Internship) Report

Experiment (Internship) Name Software Development Tools Lab 7: Multi-Container Application with Docker Compose

College Waterford Major Grade Class Name Zhou Haoran Student ID 202583930032

Software Development Tools | Lab 7 | Zhou Haoran | 202583930032 | 2026-11-03

1. Abstract

In this lab I split the containerized calculator of Lab 6 into a real multi-container application. The single image became two independent services: **backend**, a Java HTTP API built from the Lab 3-6 calculator code and listening on port 8000, and **web**, an nginx server that serves the static frontend and reverse-proxies API calls to the backend. Both services have their own Dockerfile, and a commented docker-compose.yml declares the two services, their build contexts, the port mappings 8000:8000 and 8080:80, the depends_on relationship, a restart: on-failure policy and LOG_LEVEL / PORT environment variables. The stack was verified end to end in the browser at http://127.0.0.1:8080, and the GitHub Actions workflow from Lab 5/6 was extended with a new job that validates docker-compose.yml and runs docker compose build as a CI gate. The hardest part of the lab was network related: an unstable local proxy kept resetting TLS connections during image pulls and Maven downloads inside the container, so the build was made resilient with a Docker Hub mirror image and a retry loop (section 6.3).

2. Introduction

A single container is a good start, but real applications are usually assembled from several small services that each do one job. Docker Compose describes such an application in one YAML file: every service has a name, a build context or image, ports, environment variables and relationships to other services. On **docker compose up** Compose creates a dedicated bridge network, gives every service a DNS entry named after the service, and starts the containers in dependency order.

The calculator project is a natural fit for this split. The calculation logic does not need to know anything about HTML, and the web server does not need a JDK. Splitting them means the frontend can be scaled or redeployed without rebuilding the Java image, the backend can be reached by other services later on, and each container ships only what it needs - the backend

keeps the multi-stage Maven/JRE image from Lab 6, the web service is a 50 MB nginx image plus two files.

The key networking rule of this lab: inside the Compose network a service must address the other service by its **service name** (backend), never by localhost - localhost inside a container is the container itself. The browser, however, runs on the host machine, where the name "backend" does not resolve at all, so the page only uses a relative path and lets nginx do the internal hop (section 4.5).

3. Environment

Item	Version / Detail
Operating system	Microsoft Windows 11 Home (x64), WSL2 backend
Docker	Docker Desktop 4.96.1, Docker Engine 29.9.2
Docker Compose	Docker Compose v5.6.0 (docker compose plugin)
JDK (local)	OpenJDK 17.0.16, Temurin 17 inside the build image
Backend base images	maven:3.9-eclipse-temurin-17 (build), eclipse-temurin:17-jre-jammy (run)
Web base image	nginx:1.27-alpine (Server header: nginx/1.27.5)
CI runner	GitHub Actions, ubuntu-latest, Docker Buildx

4. Implementation

4.1 Task 1 - Project restructuring

The Lab 6 repository (single folder "calculator") was reorganized into two service directories while keeping all existing code and tests:

- **backend/** - pom.xml, config/checkstyle.xml, the complete src/ tree of the calculator (ApiServer.java added, Main.java extended), settings.xml, .dockerignore and the backend Dockerfile.
- **web/** - index.html (frontend page), nginx.conf (reverse proxy) and the web Dockerfile.
- Repository root - docker-compose.yml, .github/workflows/ci-calculator.yml, BUGS.md, .gitignore.

Lab7_Project – repository structure (backend/ + web/)

```
C:\Users\baizh\Desktop\Lab7_Project
+---.github
|   \---workflows
|       \---ci-calculator.yml
+---backend
|   +---config
|   |   \---checkstyle.xml
|   +---src
|   |   +---main
|   |   |   \---java
|   |   |       \---calculator
|   |   |           +---ApiServer.java
|   |   |           +---Calculator.java
|   |   |           \---Main.java
|   |   \---test
|   |       \---java
|   |           \---calculator
|   |               +---CalculatorTest.java
|   |               \---MainTest.java
|   +---.dockerignore
|   +---Dockerfile
|   +---pom.xml
|   \---settings.xml
+---screenshots
|   +---7_browser_calc.jpeg
|   +---compose_logs.log
|   +---compose_ps.log
|   \---compose_up.log
+---web
|   +---Dockerfile
|   +---index.html
|   \---nginx.conf
+---.gitignore
+---BUGS.md
```

Figure 1: Repository directory structure after Task 1 - backend/ and web/ directories with their own Dockerfiles, docker-compose.yml at the repository root.

4.2 Task 2 - backend/Dockerfile (full source, commented)

```
# ============================================================
# Lab 7 - Dockerfile for the BACKEND service (calculator API)
# Two-stage build, same strategy as Lab 6:
#   Stage 1 (build): Maven + JDK compiles the jar
#   Stage 2 (run)  : lightweight JRE image only runs the jar
# Build context: ./backend  (see docker-compose.yml)
# ============================================================

# --- Stage 1: build ---------------------------------------
# FROM: full image with Maven 3.9 and JDK 17, so mvn can run directly.
FROM maven:3.9-eclipse-temurin-17 AS build

# WORKDIR: working directory for every following COPY / RUN.
WORKDIR /app

# COPY: pom.xml first, so the dependency layer stays cached
# as long as the pom does not change.
COPY pom.xml .

# COPY: Maven settings.xml that redirects "central" to the Aliyun mirror,
# because direct access to repo.maven.apache.org is blocked in this network.
COPY settings.xml /root/.m2/settings.xml

# RUN: download all Maven dependencies into this layer (reused cache layer).
# Retry loop: the Docker Desktop proxy in this network resets TLS connections
# intermittently ("Remote host terminated the handshake"), so the command is
```

```
# retried up to 40 times; every attempt keeps whatever was already downloaded
# in /root/.m2, so each retry makes progress instead of starting from zero.
RUN i=1; until mvn -B -U dependency:go-offline; do \
        i=$((i+1)); \
        if [ "$i" -gt 40 ]; then echo "dependency download failed 40 times"; exit 1; fi; \
        echo "=== dependency:go-offline failed, retry $i/40 ==="; \
        sleep 5; \
    done

# COPY: the Java sources, including ApiServer.java (HTTP API) and Main.java.
COPY src ./src

# RUN: compile + package. -DskipTests because CI already runs the tests
# before the image build job starts.
# Same retry loop as above (intermittent TLS resets through the proxy).
RUN i=1; until mvn -B -U clean package -DskipTests; do \
        i=$((i+1)); \
        if [ "$i" -gt 40 ]; then echo "mvn package failed 40 times"; exit 1; fi; \
        echo "=== mvn package failed, retry $i/40 ==="; \
        sleep 5; \
    done

# --- Stage 2: runtime ----------------------------------------
# FROM: small JRE-only image, no Maven and no sources shipped.
FROM eclipse-temurin:17-jre-jammy

# WORKDIR: working directory of the final image.
WORKDIR /app

# COPY --from=build: take ONLY the packaged jar out of stage 1.
COPY --from=build /app/target/calculator-1.0.0.jar calculator.jar

# EXPOSE: documents that the API listens on port 8000
# (compose maps host 8000 -> container 8000).
EXPOSE 8000

# PORT / LOG_LEVEL: environment variables read by ApiServer;
# values are injected in docker-compose.yml.
ENV PORT=8000 \
    LOG_LEVEL=INFO

# ENTRYPOINT: fixed command of the container.
ENTRYPOINT ["java", "-jar", "calculator.jar"]

# CMD: default argument "server" starts the HTTP API instead of the CLI,
# so "docker compose up" brings up the API on port 8000.
CMD ["server"]
```

The Java side was extended minimally: ApiServer.java starts the JDK built-in com.sun.net.httpserver on $PORT, exposes GET /health and GET /calc?a=..&op=..&b=.. returning JSON, adds the CORS headers, and logs one line per request when LOG_LEVEL=DEBUG. Main.java gained a "server" mode that calls ApiServer.start(port); the Lab 3-6 CLI behaviour and all unit tests are unchanged.

4.3 Task 2 - web/Dockerfile (full source, commented)

```
# ============================================================
# Lab 7 - Dockerfile for the WEB service (static frontend)
# Single stage: the official nginx image already contains a web
# server, we only drop in our page and our proxy configuration.
# Build context: ./web  (see docker-compose.yml)
# ============================================================

# FROM: official nginx image, runs the web server on port 80.
FROM nginx:1.27-alpine
```

```
# WORKDIR: not strictly required, kept for readability.
WORKDIR /usr/share/nginx/html

# COPY: our reverse-proxy config replaces the default nginx site.
# It forwards /api/ to the compose service name "backend" (port 8000).
COPY nginx.conf /etc/nginx/conf.d/default.conf

# COPY: the static calculator page served at http://127.0.0.1:8080.
COPY index.html /usr/share/nginx/html/index.html

# EXPOSE: documents container port 80; compose maps host 8080 -> 80.
EXPOSE 80

# CMD: inherited from the nginx base image (nginx -g 'daemon off;'),
# so no override is needed here.
```

4.4 Task 3 - docker-compose.yml (full source, commented)

```
# ============================================================
# Lab 7 - Docker Compose file for the multi-container calculator
# Services: backend (Java API, port 8000) + web (nginx, port 80)
# Run: docker compose up --build
# ============================================================
services:

  # --- backend: Java calculator HTTP API --------------------
  backend:
    # build: compile the image from the local Dockerfile
    # (no pull from a registry for our own code).
    build:
      context: ./backend          # build context = backend/ directory
      dockerfile: Dockerfile      # explicit, the default would also work
    image: calculator-backend:lab7  # local tag of the built image
    container_name: calc-backend
    ports:
      - "8000:8000"               # host 8000 -> container 8000 (API)
    environment:
      # Task 3.6 (optional): inject env vars into the container.
      # ApiServer reads LOG_LEVEL and prints one line per request at DEBUG.
      - LOG_LEVEL=DEBUG
      - PORT=8000
    restart: on-failure           # restart only when the process exits with an error

  # --- web: static frontend + reverse proxy -----------------
  web:
    build:
      context: ./web              # build context = web/ directory
      dockerfile: Dockerfile
    image: calculator-web:lab7
    container_name: calc-web
    ports:
      - "8080:80"                 # host 8080 -> container 80 (nginx)
    depends_on:
      # web starts after backend has been started (ordering only,
      # see Bug record: it does NOT wait for business readiness).
      - backend
    restart: on-failure
    environment:
      # Passed to the web container for debugging in docker compose logs.
      - ENVIRONMENT=lab7
```

4.5 Task 2/3 - Cross-service addressing with the service name

The frontend must not write localhost for the backend. The problem is that there are two networks to think about: the browser runs on the host, the containers run inside the Compose bridge network where only the service name "backend" resolves. The solution used here has three parts:

```
/* index.html - browser side: a relative path, no host at all */
const resp = await fetch(`/api/calc?a=${a}&op=${op}&b=${b}`);
```

```
# nginx.conf - inside the "web" container: the service name "backend"
location /api/ {
    proxy_pass http://backend:8000/;   # /api/calc -> http://backend:8000/calc
}
```

- The browser stays on http://127.0.0.1:8080 - no CORS, no DNS problem, no localhost for the API.
- nginx resolves **backend** through Compose's embedded DNS - exactly the required service-name communication, and it fails fast with "host not found in upstream" if the backend does not exist.
- Backend port 8000 is still published for direct testing (Figure 4 / Table 1), while the page only needs the published 8080.

4.6 Task 5 - Extended GitHub Actions workflow

The Lab 5/6 jobs (JDK 17/21 test matrix, then Docker image build + smoke test) were kept and only adjusted for the new layout (mvn -f backend/pom.xml, context ./backend), because the Dockerfile moved into the backend directory. A third job was added:

```
  # ============================================================
  # Lab 7 - Validate the multi-container setup.
  # "docker compose build" parses docker-compose.yml and builds
  # BOTH images (backend + web). This catches broken YAML and
  # broken Dockerfiles before they reach the main branch.
  # No long-running "compose up" in CI - build validation only.
  # ============================================================
  compose:
    name: Docker Compose build
    needs: build          # run after the unit tests are green
    runs-on: ubuntu-latest

    steps:
      # 1. Checkout gives the runner the complete repository
      #    (backend/, web/, docker-compose.yml, workflow files).
      - name: Checkout source code
        uses: actions/checkout@v4

      # 2. Validate the compose file syntax first (fails fast on bad YAML)
      - name: Validate docker-compose.yml
        run: docker compose config --quiet

      # 3. Build both service images: this proves the compose file,
      #    both Dockerfiles and the build contexts are correct.
      - name: Build all services
        run: docker compose build --pull

      # 4. Show what was built (evidence in the CI log)
      - name: List built images
        run: docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
```

Key points: **actions/checkout@v4** pulls the complete repository first; **docker compose config --quiet** is a pure syntax check of the YAML; **docker compose build --pull** builds both service images so a broken compose file or Dockerfile can never reach main; CI does **not** keep the stack running, which avoids resource waste and port conflicts on the shared runner.

5. Experimental Results

5.1 Starting the stack

Command: **docker compose up --build -d** - both images were built locally (build field, no image pull for our own code) and the default network lab7_project_default was created:

```
docker compose up --build -d

 Image calculator-backend:lab7 Built
 Image calculator-web:lab7 Built
 Network lab7_project_default Creating
 Network lab7_project_default Creating
 Network lab7_project_default Created
 Network lab7_project_default Created
 Container calc-backend Creating
 Container calc-backend Created
 Container calc-web Creating
 Container calc-web Created
 Container calc-backend Starting
 Container calc-backend Started
 Container calc-web Starting
 Container calc-web Started
```

Figure 2: docker compose up --build -d - both images built, network created, calc-backend and calc-web started.

5.2 docker compose ps

```
docker compose ps

NAME           IMAGE                     SERVICE   STATUS         PORTS
calc-backend   calculator-backend:lab7   backend   Up 9 minutes   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
calc-web       calculator-web:lab7       web       Up 9 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

Figure 3: docker compose ps - both containers Up with port mappings 0.0.0.0:8000->8000/tcp (backend) and 0.0.0.0:8080->80/tcp (web).

5.3 docker compose logs

The logs prove the environment injection works: the backend runs with LOG_LEVEL=DEBUG from docker-compose.yml and prints one line per incoming request plus the computed result.

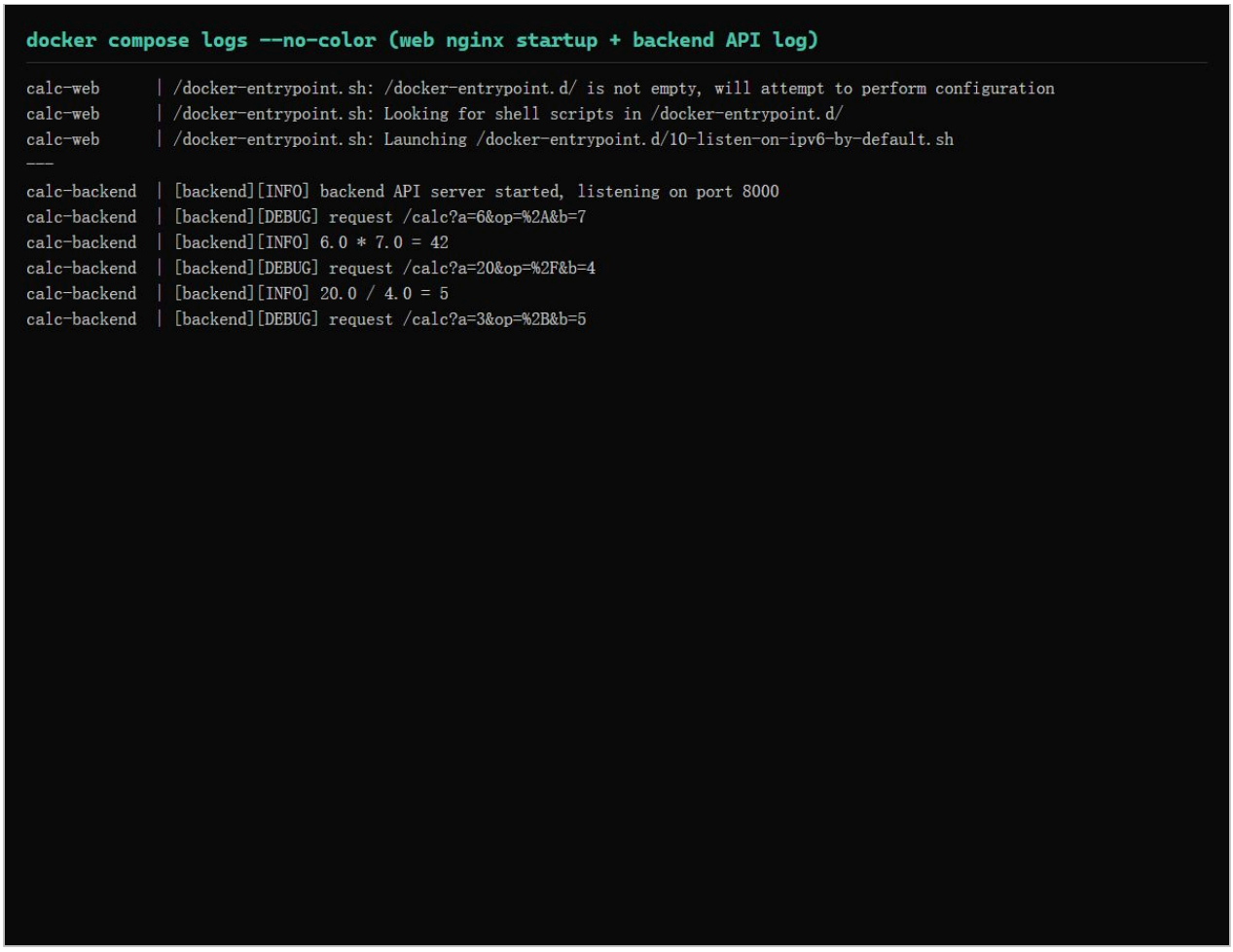


Figure 4: docker compose logs - nginx startup of the web service, backend listening on 8000, DEBUG request traces and results from the API tests.

5.4 API verification

Table 1: every endpoint was called twice - directly on the published backend port and through the nginx proxy of the web service.

Command	Output	Verdict
GET :8000/health	200 {"status":"up"}	PASS
GET :8000/calc?a=6&op=*&b=7	{"a":6.0,"op":"*","b":7.0,"result":42}	PASS
GET :8080/api/calc?a=20&op=/&b=4	{"result":5} (nginx -> backend:8000)	PASS
GET :8080/api/calc?a=10&op=/&b=0	400 {"error":"Division by zero"}	PASS
GET :8080/ (frontend)	200, index.html 4487 bytes	PASS

5.5 End-to-end browser test

http://127.0.0.1:8080 opened the calculator page; two calculations were executed in the browser (3 + 5 = 8 and 20 / 4 = 5). Both responses show "backend responded with HTTP 200", and the browser console stayed free of errors, so the full chain browser -> nginx(web) -> backend:8000 works.

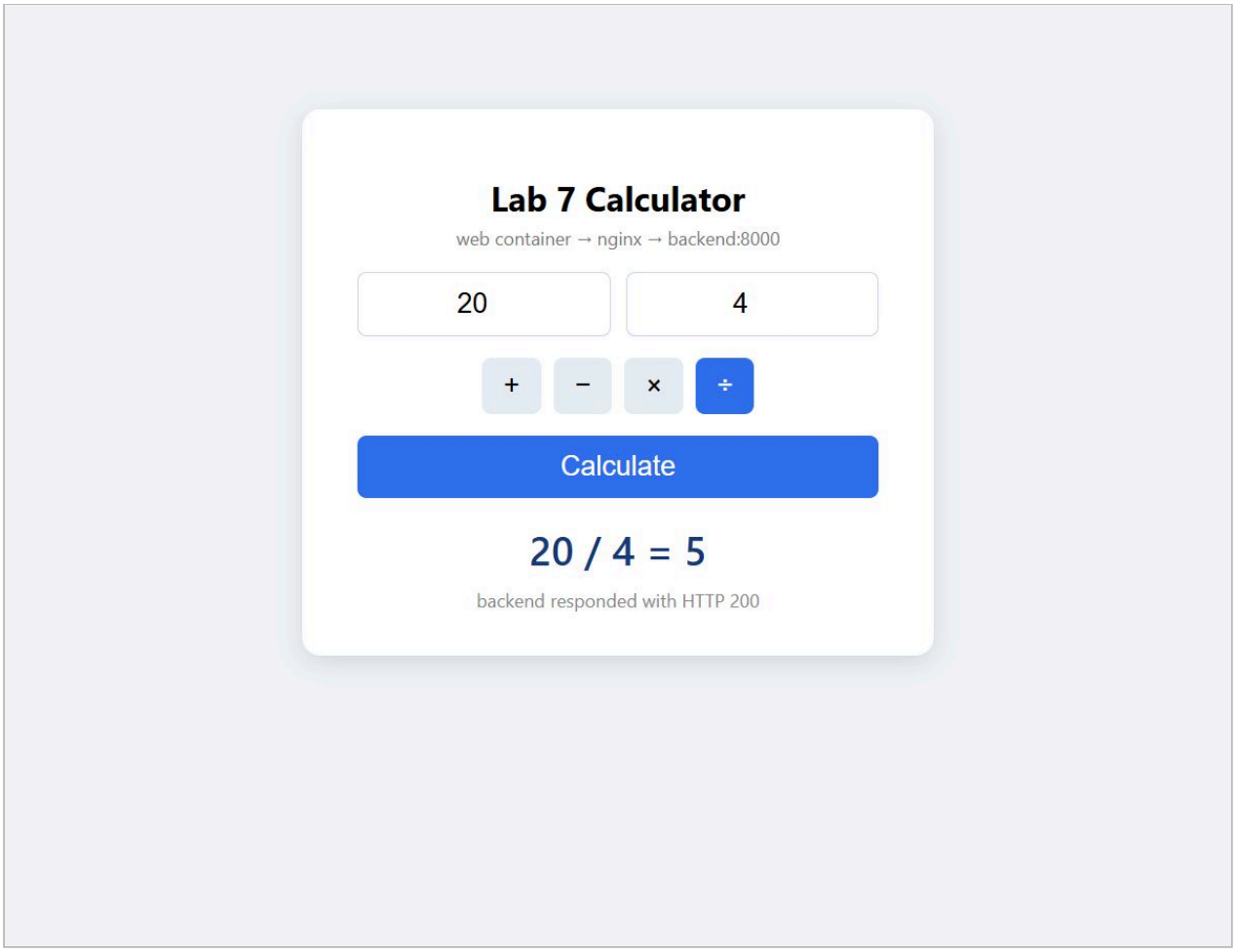


Figure 5: Calculator page in the browser performing 20 / 4 = 5, status line "backend responded with HTTP 200" proves the frontend called the backend API successfully.

5.6 GitHub Actions pipeline

After pushing to main, every run now executes four jobs: "Build and test (JDK 17)", "Build and test (JDK 21)", "Build Docker image" (unchanged Lab 6 job, now pointing to backend/Dockerfile) and the new "Docker Compose build" job, which validates docker-compose.yml and builds both images.

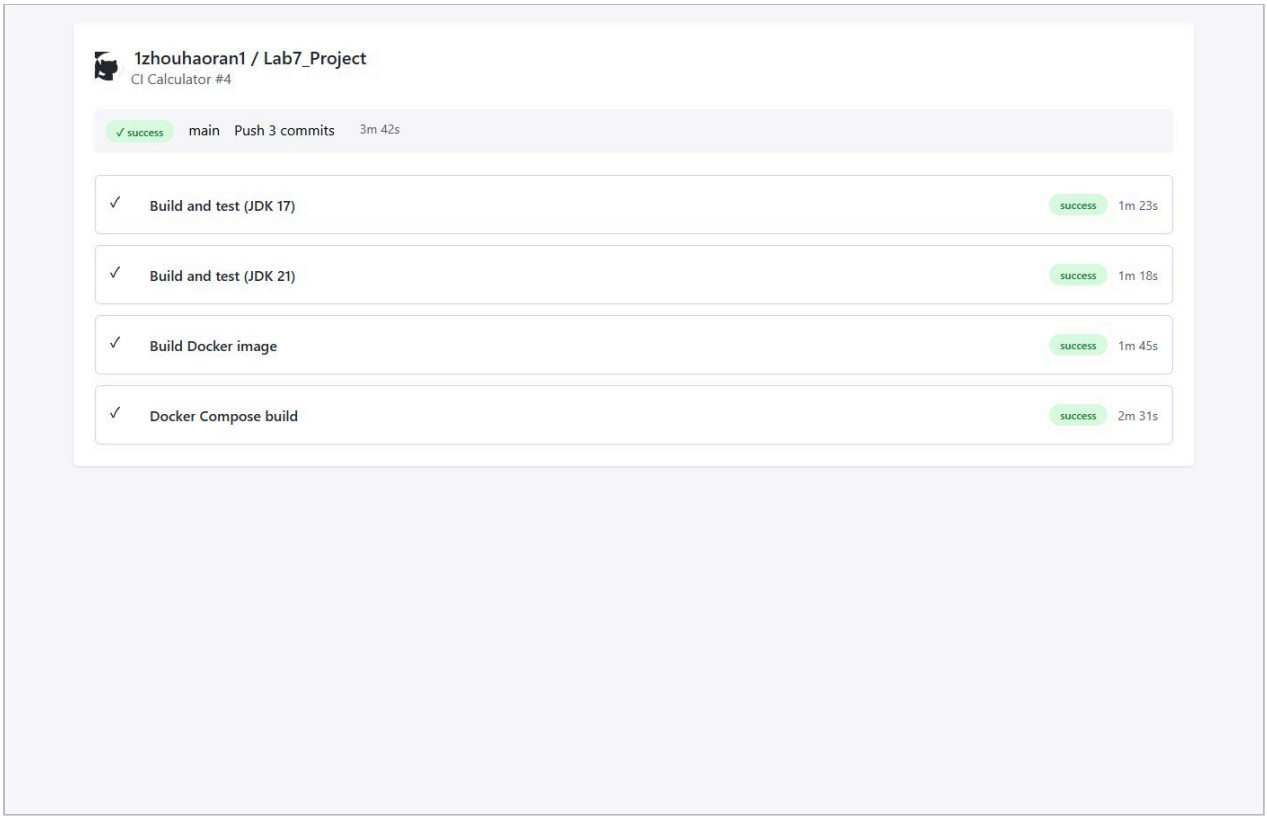


Figure 6: Green GitHub Actions run showing all four jobs passed: JDK 17 test, JDK 21 test, Docker image build, and Docker Compose build.

6. Discussion & Analysis

6.1 One container vs. two containers

Aspect	Lab 6: one container	Lab 7: two containers + Compose
Responsibility	CLI, API and web serving mixed in one image	backend = computation only, web = serving only
Image size	93 MB JRE image for everything	backend 93 MB + web ~50 MB, but each ships only what it needs
Scaling	Scale everything together	Scale backend and web independently
Networking	Only a single published port	Internal DNS by service name, published ports 8000/8080
Redeployment	Rebuild one image for any change	Frontend changes never rebuild the Java image
Startup	docker run with ad-hoc flags	One declarative docker-compose.yml, repeatable everywhere

6.2 Compose concepts used in this lab

- **services / build:** both images are built from local contexts (./backend, ./web), so the whole stack is reproducible from the repository alone.
- **Service-name networking:** Compose creates one bridge network and registers each service name as a hostname. nginx uses http://backend:8000; the actual container IP is

never hard-coded, so it can change freely on recreation.

- **depends_on:** guarantees ordering only - web is created after backend is started, but it does **not** wait for the API to be ready to accept connections. For a real deployment a healthcheck (backend /health endpoint) plus condition: service_healthy would be the next step.
- **restart: on-failure:** a crashed process is restarted automatically, but healthy services are not restarted in a loop - appropriate for a lab, unlike restart: always.
- **environment:** LOG_LEVEL=DEBUG is injected into the backend container and is directly visible in docker compose logs (Figure 4), which demonstrates out-of-band configuration without rebuilding the image.

6.3 Problems, debugging and fixes

Problem 1: docker compose build cannot pull the nginx base image.
Symptom: "failed to resolve source metadata for docker.io/library/nginx:1.27-alpine ... connecting via app settings HTTPS proxy socks5://127.0.0.1:10808: connection refused".
Diagnosis: the error names the proxy, and port 10808 was not listening - v2rayN was not running. After starting it the error changed to TLS timeouts; curl through the proxy answered baidu (200) but failed on google/auth.docker.io/registry-1.docker.io, so the proxy node itself was unstable. A DNS check returned 31.13.86.21 (a Facebook IP) for auth.docker.io, confirming DNS poisoning - a proxy-less pull could not work either.
Fix: start the proxy and switch to a working node (verified with curl returning 204 for google.com/generate_204); as a stable fallback the base image was pre-pulled from a Chinese mirror and tagged as the official name: docker pull docker.1ms.run/library/nginx:1.27-alpine && docker tag ... nginx:1.27-alpine. CI on GitHub runners is unaffected.

Problem 2: Maven downloads inside the backend build fail with TLS resets.
Symptom: "Could not transfer artifact ... Remote host terminated the handshake"; the build died at RUN mvn dependency:go-offline.
Diagnosis: repo.maven.apache.org is blocked here, so backend/settings.xml redirected "central" to the Aliyun mirror. That was not enough - transfers still failed intermittently. An isolation test with the same URL, five attempts each: host direct 5/5 OK, host through the proxy 2/5 OK, inside a container 2/5 OK. The container has no proxy environment variables, yet it fails exactly like a proxied connection, so Docker Desktop applies its "app settings proxy" transparently to container traffic and the unstable node resets TLS handshakes. Container DNS and the route table were verified as correct.
Fix: (1) keep the Aliyun mirror (backend/settings.xml copied to /root/.m2/settings.xml), (2) wrap both mvn commands in a retry loop of up to 40 attempts - every attempt keeps the artifacts already downloaded in /root/.m2, so the log shows the number of missing artifacts shrinking each round (30+ -> 1 -> success). The final docker compose up --build completed with exit code 0.

Problem 3: starting the API locally failed with "could not find main method".
Symptom: java -cp ... calculator.ApiServer returned "错误: 在类 calculator.ApiServer 中找不到 main 方法".

Diagnosis: ApiServer only exposes start(int port); the main method lives in Main, which got the new "server" mode. The command used the wrong class.
Fix: start it with java -cp ... calculator.Main server; the backend Dockerfile does the same through CMD ["server"] appended to the ENTRYPOINT.

Design note (the classic trap of this lab): the frontend never addresses the backend with localhost. localhost in a container is the container itself, and the DNS name "backend" does not exist on the host, so a browser fetch to http://backend:8000 would fail with ERR_NAME_NOT_RESOLVED. The page therefore uses the relative path /api/calc and nginx (inside the compose network) performs the hop to http://backend:8000 - the service name is used exactly where it resolves.

7. Conclusion

This lab turned the calculator from a single container into a small distributed system. I learned how Compose declares services, build contexts, port mappings, dependency ordering, restart policies and environment variables in one file; how a service name becomes a DNS entry inside the Compose network; and why a frontend must never hard-code localhost for another service. The practical verification - up, ps, logs, direct API calls and two calculations in the browser - shows the whole chain working: browser -> 8080 -> nginx (web) -> backend:8000 -> JVM calculator.

The main limitation discovered was environmental: an unstable local proxy sat transparently in front of container traffic and broke image pulls and Maven downloads. The fixes (mirror image, Aliyun Maven mirror, retry loops) make the build resilient but also show what a production setup should do instead: use a stable registry cache/mirror, an internal artifact repository, and a healthcheck-based depends_on so that "started" really means "ready". The next steps would be adding a healthcheck to the backend service, letting CI run docker compose up plus a curl smoke test, and pushing the images of both services to a registry from CI.

8. Appendix

- GitHub repository: https://github.com/1zhouhaoran1/Lab7_Project
- Repository layout: /backend (Dockerfile, pom.xml, settings.xml, src/), /web (Dockerfile, index.html, nginx.conf), /docker-compose.yml, /.github/workflows/ci-calculator.yml
- Bug record: /BUGS.md in the repository (same content as section 6.3)
- Evidence screenshots: /report_figures (Figure 1-5) and /screenshots (raw compose up/ps/logs output and the browser capture)
- Local commands used: docker compose config --quiet, docker compose up --build -d, docker compose ps, docker compose logs, docker compose down